

IL LAMPONE INFORMATICO: BRACCIO ROBOTICO



Alberto Fasulo

Liceo Lugano 1

Lavoro di Maturità

2021-2022

Informatica: Lampone informatico

Docente: Amedeo Mazzoleni

Abstract

Dopo aver scelto senza troppe difficoltà il mio lavoro di maturità, e dopo essere stato assegnato ad esso, mi rimaneva solamente da scegliere che progetto fare, le scelte, grazie alla prestanza del Raspberry Pi (che è un computer a scheda singola) ed alla sua versatilità derivate anche dal suo piccolo volume, erano moltissime. Grazie alla mia propensione alla manualità decisi di utilizzare il computer per controllare dei dispositivi, in modo analogo a come avevo già fatto con Arduino. Volevo comunque sfruttare al meglio tutte le potenzialità di questo strumento, decisi così di voler rendere il mio progetto controllabile da remoto da un qualsiasi apparecchio. Anche dopo aver ristretto maggiormente il campo le possibilità erano davvero molte, ma alla fine fu l'idea a raggiungermi e non il contrario. Un giorno a pranzo mentre mangiavo con dei miei compagni di classe, stavamo scherzando sull'idea del progetto di un mio compagno, che con Arduino voleva costruire un braccio robotico che si muoveva in sintonia con il braccio di Giovanni (l'autore del braccio), un progetto che da fare con Arduino è davvero molto impegnativo; mi venne l'idea di farla con il mio software e di comandarla da un dispositivo remoto, come il mio cellulare. Ebbi così l'idea del mio progetto di maturità, che venne ben accettata del mio docente.

Così scelta e approvata l'idea, non mancava che metterla in pratica e costruire il mio braccio robotico controllabile da remoto.

Sommario

Abstract	I
1 Introduzione.....	1
2 Mark I:	2
2.1 premessa:	2
2.2 Materiali:	2
2.3 costruzione:	3
3 Mark II:	4
3.1 materiale:.....	4
3.2 progettazione e costruzione:	4
3.3 programma del movimento:	5
3.3.1 librerie:	5
3.3.2 movimento:.....	6
3.3.3 Mark II e il suo Nervosismo:.....	7
3.3.4 movimento 1.2:	8
3.4 presa oggetti:	9
3.4.1 Programma per rilevare la distanza1:	10
3.4.2 Movimento in base alla distanza:	10
3.4.3 Movimento in base alla trigonometria:.....	12
3.5. Mark II e il suo nuovo volto:	13
3.5.1 il nuovo volto 2.0:	14
4 controllo remoto:	16
4.1 controllo da website:.....	16
4.1.1 salvataggio posizione:	17
4.2 connessione ssh:	17
5 conclusioni:	21
6 bibiloigrafia:	III

1 Introduzione

Quando vennero caricate su moodle le proposte per i lavori di maturità cominciai a scorrere le diverse proposte per trovare la più idonea a me. Me ne piacquero un paio, ma una in particolare catturò la mia attenzione, probabilmente per il suo nome *il lampone informatico*, un nome molto particolare quasi buffo; dopo aver visto questo titolo cliccai subito per saperne di più. Dopo aver visto la presentazione scelsi quasi istantaneamente questo lavoro di maturità come prima scelta. Presi questa decisione soprattutto perché il mondo dell'informatica mi appassiona molto, infatti questo lavoro di maturità si basava sulla creazione di un software mediante l'utilizzo della scheda raspberry pi, infatti anche nel mio tempo libero mi diletto nella programmazione e nella costruzione di "robot", per fare ciò utilizzavo la scheda e il software "arduino" che mi ha permesso di fare belle cose; come ad esempio il sistema di sgancio di un paracadute di un razzo a pressione. Perciò quando ho visto l'opportunità di imparare a controllare dei dispositivi con un software simile ma più potente (e con più possibilità) che era basato anche su un linguaggio di programmazione diverso colsi subito questa occasione.

2 Mark I:

2.1 premessa:

La denominazione Mark indica il livello di avanzamento, il grado di sviluppo di un macchinario¹. Uso quest'espressione per documentare lo stadio di avanzamento del mio progetto, più il numero è alto più è avanzato il mio prototipo.

Dopo aver appreso le basi della programmazione con il Raspberry pi decisi di costruire il primo prototipo in modo da capire come controllare più servomotori dalla stessa scheda.

2.2 Materiali:

I materiali usati per costruire questo prototipo sono piuttosto rudimentali almeno per il primo prototipo. Il prototipo vedeva due componenti principali(tralasciando i cavi e il Raspberry):

- 2 Servomotori sg90
- Lego

I lego formavano la struttura di base, a cui venivano collegati i servomotori. Questo primo progetto non era proprio un braccio, era più che altro una torre che girava con un servomotore sopra creato allo scopo, come già detto, di prendere dimestichezza sul controllo di essi tramite il Raspberry.

Prima di proseguire con il progetto è opportuno spiegare cosa sono i servomotori.

A **servomotor** is a rotary actuator or linear actuator that allows for precise control of angular or linear position, velocity and acceleration. It consists of a suitable motor coupled to a sensor for position feedback. It also requires a relatively sophisticated controller, often a dedicated module designed specifically for use with servomotors.²

Ovvero, I servomotori sono degli speciali motori che permettono all'utente di scegliere l'angolazione a cui ruotarli. Ciò deriva dal fatto che i servomotori sono composti da due parti distinte, una è il motore con i propri ingranaggi, mentre la seconda è il modulo di controllo, che traduce i segnali che arrivano dal computer e li tramuta nella posizione da dare al motore, come si può ben vedere dall'immagine a fianco³.

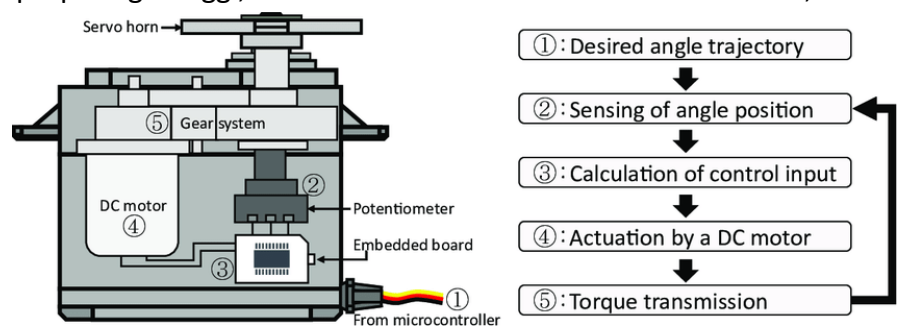


Figura 1: Rappresentazione servomotore

¹ [https://it.wikipedia.org/wiki/Mark_\(modello\)](https://it.wikipedia.org/wiki/Mark_(modello))

² Definizione da <https://en.wikipedia.org/wiki/Servomotor>

³ https://www.researchgate.net/figure/Schematic-of-an-RC-servo-motor_fig1_328910852

Gli utilizzi di questi specifici motori sono molti, come ad esempio: la robotica, l'automazione,...

Io ho utilizzato questi motori perché mi permettono il controllo dell'angolazione delle singole giunture del braccio, analogamente ad un gomito umano che si piega ad una determinata angolazione.

2.3 costruzione:

Tornando al Mark 1, questo primo prototipo era una costruzione a forma di torre reticolata a base quadrata alta una decina di centimetri e di larghezza circa cinque. Era costituita da lego. Essa ruotava su se stessa orizzontalmente grazie ad un servomotore posto alla base che fungeva anche da sostegno. Sulla sommità di questa torre era posto un quadrato di lego a cui ho attaccato tramite della colla a caldo un secondo servomotore che rappresenta il braccio. Tutto ciò è attaccato su una pedana di lego che funge da base. Sia nella costruzione che nella programmazione del Mark 1 non ho avuto grandi problemi, dopo aver capito come controllare dapprima solamente un led ed in seguito i servomotori tramite i pin GPIO (spiegati meglio in seguito) questo braccio ha cominciato a funzionare discretamente. L'unico problema che ho riscontrato è stato l'input della tastiera, poiché come libreria per ricevere l'input dalla tastiera avevo intenzione di utilizzare la libreria di Python *keyboard*, ma essa non è supportata da Linux che è il sistema operativo del Raspberry pi, per risolvere questo problema ho solamente sostituito la libreria con un'altra libreria di Python, *pygame*, essa è supportata e mi ha permesso di rilevare i tasti premuti sulla tastiera.

Questi due servomotori venivano mossi con la tastiera del computer tramite un semplice codice di una decina di righe. Sfortunatamente il codice di questo primo braccio è andato perso dopo un incidente con il Raspberry pi che portò la rottura della scheda di memoria. Questo più che il mio primo tentativo di costruire un braccio controllabile da remoto con il Raspberry pi è stato un primo approccio alle meccaniche di programmazione, e di controllo, di questo software a me sconosciuto.

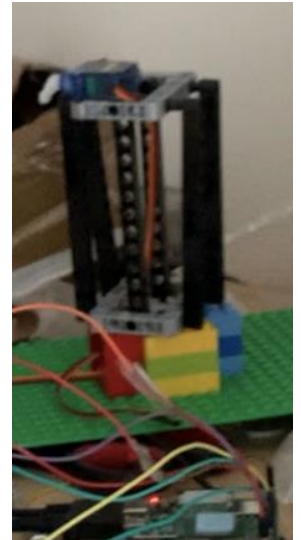


Figura 2: Torre reticolata

3 Mark II:

3.1 materiale:

Dopo aver imparato a confrontarmi con un nuovo software ed aver compreso le meccaniche di base cominciai a costruire il mio primo vero prototipo. Grazie al “Mark I” avevo compreso che i servomotori usati in questo primo prototipo non potevano essere usati per sviluppare un braccio più grande ed efficace. Arrivai a questa conclusione per due motivi, il primo consiste nella scarsa forza che un motore può esercitare, infatti un servomotore modello sg90 era in grado di sollevare un kilogrammo a un centrimetro di distanza perciò a dieci centimetri sarebbe riuscito ad alzare solamente 100 grammi, troppo poco per un prototipo con lo scopo di sollevare oggetti. Il secondo problema era la forma del motore, poiché il motore aveva solamente un lato su cui era presente un ingranaggio da poter controllare e ciò rendeva difficile attaccarci degli “avambracci”, poiché il punto di aggancio doveva potersi muovere per permettere la scelta dell’angolazione a cui muovere l’avambraccio, cosa impossibile con gli sg90 per via del singolo punto di rotazione. Così sono andato su internet e ho trovato dei motori che facevano a caso mio, sono dei motori con due punti d’aggancio, sono i “Robot servo 25kg RDS3225”, li ho trovati su Aliexpress e ne ho comprati 6⁴. Insieme ai servo motori venivano venduti anche dei supporti di alluminio che permettono di usarli per costruire dei robot.

3.2 progettazione e costruzione:

Quando finalmente sono arrivati mi sono messo subito a costruire il primo vero prototipo del braccio, dopo averli assemblati e aver provato a muoverli singolarmente per vedere se avevano differenze da quelli usati in precedenza, mi sono messo a disegnare il braccio con le misure migliori. Ho optato per un braccio che ha il primo pezzo leggermente più corto della seconda metà; infatti i due avambracci misurano 19 e 21 centimetri. Dopo aver assemblato, provato i motori e abbozzato il braccio non mi rimaneva altro che costruirlo. Per costruire i due avambracci ho optato inizialmente per l’utilizzo dei lego, come avevo fatto con il Mark I, ma è stato un fallimento, allora ho pensato ad altre soluzioni, finché non ho trovato quella più comoda e semplice: utilizzare dei listelli di legno. Sono andato allora in garage e lì ho trovato dei listelli di legno che avevano le stesse misure di spessore e di larghezza dei supporti dei servomotori, così ho preso il seghetto alternativo e ho tagliato i due pezzi della lunghezza che mi serviva. Sono allora tornato in camera dove ho iniziato ad assemblare il Mark II. Per fare i primi test ho usato solamente tre motori tralasciando la pinza. Ho preso una breadboard, che è una specie di griglia che permette di fare i collegamenti tra scheda e utilizzatori in modo più facile e ho collegato i motori alla breadboard e quest’ultima al Raspberry pi. I servomotori hanno tre cavi, uno è il polo positivo, colorato di rosso, il polo negativo colorato di marrone e il segnale colorato di arancione, i primi due devono essere sempre collegati, mentre il segnale no. Il collegamento tra un qualsiasi utilizzatore e la scheda madre riesce grazie ai GPO pin, essi infatti consentono al Raspberry pi di portare corrente a un

⁴ <https://www.aliexpress.com/item/32907625266.html>

determinato circuito od utilizzatore. Essi non hanno però solo la capacità di mandare un costante flusso di elettricità e riprenderla nella terra, ma sono anche in grado di mandare dei segnali elettrici con differenti frequenze, per permettere il controllo di dispositivi (come dei servomotori), bensì persino di ricevere segnali dai sensori e di elaborarli tramite un programma. Perciò come già detto questa scheda mi permette di controllare con facilità i miei motori.

Un'altra cosa positiva del Raspberry pi è che posso controllarlo dal computer, ovvero io, tramite la tastiera collegata al computer, posso controllare la corrente in uscita e in entrata da tutti i pin che lo permettono. Il polo positivo l'ho connesso (tramite breadboard e cavi) al pin 2 che mi fornisce una corrente di 5 Volt d.c., i miei motori funzionano con un voltaggio compreso da 4,8 a 6,8 Volt d.c., questo pin supporta tutte le richieste di corrente da parte di tutti i pin. Il polo negativo del motore va invece semplicemente (sempre tramite la breadboard) connesso alla terra, in questo caso il pin 38.

Il segnale, quindi il cavo arancione, invece è più complicato, esso va connesso a uno dei pin che possono essere controllati, quei pin il cui output può essere controllato dal Raspberry pi. Questo processo l'ho fatto con tutti i primi tre servomotori che ho usato.

Rimaneva comunque un problema, il braccio non stava in piedi da solo, e per ovviare a questo problema ho forato la scrivania e ho avvitato i supporti dei motori nel bordo della scrivania. Dopo aver collegato tutti i motori e fissato il braccio sono passato allo sviluppo del codice.

3.3 programma del movimento:

Dopo aver approfondito un po' il funzionamento con i primi 3 motori ho deciso di installarne altri due sulla punta per formare una pinza e qui ho elaborato il primo vero e proprio codice per il movimento del braccio.

3.3.1 librerie:

La prima libreria che ho usato per il movimento del braccio, o meglio per il movimento dei motori, è la libreria RPi.GPIO⁵. Questa libreria, pubblicata da un ricercatore del MIT, serve per facilitare il controllo dei pin che sono presenti sul Raspberry Pi, i pin GPIO, quelli citati in precedenza. Tramite questa libreria infatti si può infatti accedere al funzionamento dei pin con poche linee di codice. Come si può vedere dall'immagine qui a destra infatti dopo aver dichiarato la libreria bisogna però comunque fare un paio di operazioni, per prima cosa bisogna impostare il conteggio dei pin ovvero che numero si assegna ad ogni pin (linea 7); poi bisogna impostare il pin, ovvero bisogna dichiarare il numero del pin che si vuole usare e dichiarare se questo pin deve fare da "OUTPUT" o da "INPUT", cioè se il compito del pin è ricevere o dare un segnale elettrico. E dopo aver

```
1 import RPi.GPIO as GPIO
2 import time
3 import pygame
4 pygame.init()
5 clock = pygame.time.Clock()
6 pygame.display.set_mode((300, 300))
7 #inizializzazione pin
8 GPIO.setmode(GPIO.BCM)
9 GPIO.setup(17,GPIO.OUT)
10 GPIO.setup(23,GPIO.OUT)
11 GPIO.setup(24,GPIO.OUT)
12 GPIO.setup(25,GPIO.OUT)
13 GPIO.setup(8,GPIO.OUT)
14
15
16
17 GPIO.setwarnings(False)
18
19 servo1 = GPIO.PWM(17,50)
20 servo2 = GPIO.PWM(23,50)
21 servo3 = GPIO.PWM(24,50)
22 servo4 = GPIO.PWM(25,50)
23 servo5 = GPIO.PWM(8,50)
```

Figura 3

⁵ <https://pypi.org/project/RPi.GPIO/>

dichiarato il pin si è pronti per controllarlo. Siccome però i miei pin devono controllare anche dei servomotori, essi necessitano un segnale con una determinata frequenza per muoversi, nel mio caso tra i 50 e i 330 hertz. Ho quindi dovuto, tramite le linee 19-23, dichiarare la frequenza che i pin dovevano emettere, in questo caso 50 Hz. Come si può vedere grazie a questa libreria si può accedere con facilità ai pin.

Per questo primo codice ho usato anche un'altra libreria, poiché dovevo anche decidere come muovere i motori e quindi cosa far fare ai pin del segnale.

Per fare ciò volevo utilizzare la tastiera, sfortunatamente come già detto la libreria "keyboard" non è disponibile su Linux, per ovviare a questo problema ho utilizzato la libreria Pygame⁶. Questa libreria, creata per lo sviluppo di videogiochi, permette di elaborare i segnali ricevuti dalla tastiera tramite delle linee di codice e usarli per, ad esempio, muovere un personaggio all'interno del proprio videogioco. Io ho utilizzato questa funzionalità, ma al posto di muovere un braccio formato da pixel, ne muovo uno di legno.

Questa libreria, come la precedente, ha un'ampia documentazione, perciò non ho avuto tante difficoltà ad usarla, forse anche perché l'avevo già utilizzata in passato per lo sviluppo di un gioco. Questa libreria dopo essere stata chiamata deve anche essere inizializzata (linea 4), bisogna anche impostare uno schermo sul quale teoricamente va costruito il gioco.

3.3.2 movimento:

Dopo aver dichiarato le librerie posso finalmente passare a scrivere il codice per muovere i motori. Il codice per muovere i motori con la tastiera è abbastanza monotono, l'ho ripreso dal primo programma del Mark I, e l'ho solamente esteso a più motori.

Anche durante lo sviluppo di questo programma non ho trovato grandi problemi, in sé solo uno.

Dopo aver fatto partire i motori con il comando adeguato sono potuto passare alla costruzione della mia funzione di movimento, essa è indicata qui sotto.

Come prima cosa dichiaro sei variabili, una per ogni motore, e una per la scelta di essi, le prime linee dopo le variabili sono le linee standard per far funzionare la pygame, queste le ho prese dalla scheda di teoria, la mia parte arriva dopo. In questo caso è mostrato solamente il movimento di un motore, ma il metodo è analogo per tutti, tranne che per la pinza, poiché lì i motori si muovono insieme ma in direzione opposta. Il controllo è suddiviso in tre parti; la prima è la scelta del motore, per scegliere che motore muovere ho optato per selezionarlo con la tastiera, per fare ciò bisogna come prima cosa utilizzare il comando ".KEYDOWN" che permette al software di capire se un pulsante è stato premuto o no; dopodiché bisogna usare il comando della linea 54, esso dice che se un bottone è premuto, e come in questo caso è il tasto 1, aggiorna la variabile stato. Questo processo è presente per tutti i motori e la pinza. In seguito, dopo che l'utente ha selezionato il motore da muovere, c'è un if statement che controlla il valore della variabile di selezione del motore e "attiva" il controllo di quel motore. In seguito ci sono altri due condizioni che permettono lo spostamento dell'albero del motore verso destra o sinistra; esse, in modo analogo alla scelta del motore, guardano se l'utente preme la freccia sinistra o destra e cambiano una variabile di conseguenza, ovvero la variabile specifica per il movimento di ogni singolo motore.

⁶ <https://www.pygame.org/>

Questa variabile andrà poi inserita in secondo luogo all'interno del comando per spostare i motori.

Rimane ancora un condizione, visibile dal codice, essa ha il compito di non far crescere o decrescere troppo la nostra variabile per mantenerla sempre all'interno dei valori accettati dal motore, fa ciò reimpostando la variabile al valore massimo se lo supera o a quello minimo se diventa più piccola. Questo procedimento per la scelta e il movimento del motore avviene in modo quasi analogo per ogni motore, cambiano solo i nomi delle variabili. La variabile, il cui valore cresce o decresce grazie all'utente, deve essere elaborata e spedita al motore, ciò viene fatto tramite queste linee di codice a fianco.

```

35 def tastiera():
36     b = 9.5
37     c = 9.5
38     d = 7
39     e = 9
40     f = 8
41     #variabile per la scelta del motore
42     stato = 1
43     running = True
44     #schifo di pygame iniziale
45     while running:
46         for event in pygame.event.get():
47             if event.type == pygame.QUIT:
48                 running = False
49                 pygame.quit()
50
51
52
53         if event.type == pygame.KEYDOWN:
54             if event.key == pygame.K_1:
55                 stato = 1
56                 #movimento motore 1
57                 if stato == 1:
58                     if event.key == pygame.K_LEFT:
59                         b = b + 0.3
60                         if b > 12:
61                             b = 12
62
63
64                     elif event.key == pygame.K_RIGHT:
65                         b = b - 0.3
66                         if b < 2:
67                             b = 2

```

Per prima cosa si dichiara quale motore deve accettare quale variabile, o meglio quale pin deve mandare quale segnale. Questo si fa con il codice scritto tra le righe 19-23, in cui si assegna a una variabile un pin e una frequenza. Dopodiché si collega ad ogni servo il comando dello spostamento, in questo caso è ChangeDutyCycle, che permette di prendere il valore di una variabile e di mandarlo ad elaborare all'utilizzatore. I valori delle variabili del movimento, che oscillano tra 2 e 12 a piacimento, vengono tradotte dal comando ChangeDutyCycle e mandate al motore per farlo muovere. A questo punto ho trovato il mio primo, di molti, problemi, ovvero che quando premevo un tasto il codice aggiornava la coordinata, facendola crescere di 0.3 e il motore ruotava, ma poi per muovere ancora il motore bisognava rilasciare il pulsante e premerlo nuovamente. Questo problema non è stato troppo "problematico", infatti dopo che ho capito perché lo faceva l'ho subito risolto. Esso derivava dal fatto che la pygame non si riaggiornava, si aggiornava solamente quando un evento succedeva e non ogni intervallo determinato. Ma dopo aver messo l'aggiornamento di pygame a brevi intervalli il problema è stato risolto.

```

117 servo1.ChangeDutyCycle(b)
118 servo2.ChangeDutyCycle(c)
119 servo3.ChangeDutyCycle(d)
120 servo4.ChangeDutyCycle(e)
121 servo5.ChangeDutyCycle(f)

```

Figura 5

Figura 4

3.3.3 Mark II e il suo Nervosismo:

Siamo arrivati a parlare del primo vero problema che ho affrontato, un problema di Nervosismo. Il codice con un po' di fatica sembrava ora funzionare senza troppi problemi e mi permetteva di controllare singolarmente ogni motore e di muovere il braccio fino a fargli prendere degli oggetti, ma vedevo che ogni volta che i motori non venivano mossi

diventavano nervosi, mi spiego meglio, quando un motore era “a riposo” esso non era del tutto fermo, ma il suo albero oscillava di un po’ di gradi da una parte e dall’altra, rendendo il braccio instabile e perennemente in leggero movimento.

Uso il termine nervoso, sia perché si addice in quanto i motori sembrano come una persona impaziente, ma soprattutto perché questo è un problema comune, in inglese si chiama “jitter” che tradotto letteralmente significa nervosismo. Diciamo che non ho mai risolto veramente questo problema, e penso che fino alla fine sia stato uno dei problemi più difficili da affrontare, ho passato quasi tutto il primo mese delle vacanze a cercare invano di risolvere questo problema, ho passato ore e ore a leggere la documentazione, passare da forum a forum per cercare una risposta, guardare tutti i video che spiegavano come usare questa libreria, ho persino visitato la pagina due di google, ma tutto invano.

Ognuno attribuiva una diversa causa al problema, chi la frequenza sbagliata, chi la mancanza di condensatori per rendere l’impulso più fluido e senza interruttori e molto altro.

Alla fine mi sono arreso e ho cercato una soluzione alternativa al mio problema, ho optato per andare alla radice del problema e cambiare quella, sono risalito al fatto che il problema derivava dalla libreria che usavo, o almeno così pensavo, così sono andato su internet e ho cercato una libreria con cui sostituire quella problematica. E l’ho trovata. E sono riuscito a risolvere il problema di nervosismo.

3.3.4 movimento 1.2:

Quella che sembrava la migliore per il movimento dei servo motori è la libreria `gpiozero`⁷, un'altra libreria, che come la precedente, serviva per facilitare il controllo dei pin del Raspberry pi. Anche questa aveva un’ampia documentazione, ma a differenza dell’altra aveva, a mio avviso, parecchi comandi in più. Inanzitutto da questa libreria si può importare “servo” una funzione che ti permette di controllare i tuoi servomotori con molta più semplicità. Dopo aver riscritto l’intero codice con la nuova libreria e averlo fatto partire ho visto con mio dispiacere e disappunto che il nervosismo, sebbene diminuito, era ancora presente, ma leggendo la documentazione di questa libreria ho visto un comando che ha risolto tutti i miei problemi, il comando `PiGPIOFactory`.

Questo comando ogni volta che partiva il programma faceva ripartire i motori e i pin e impostava la frequenza perfetta per comunicare, grazie a questo comando il mio problema del nervosismo è sparito completamente. Come si può vedere (nell’immagine accanto) il codice non è cambiato molto, tranne per le librerie e per il

```
2 from gpiozero import Servo
3 import time
4 from gpiozero.pins.pigpio import PiGPIOFactory
5
6
7 import pygame
8 pygame.init()
9
10 factory= PiGPIOFactory()
11 servo1_pin = 27
12 servo2_pin = 12
13 servo3_pin = 24
14 servo4_pin = 25
15 servo5_pin = 8
16
17 servo1 = Servo(servo1_pin, pin_factory=factory)
18 servo2 = Servo(servo2_pin, pin_factory=factory)
19 servo3 = Servo(servo3_pin, pin_factory=factory)
20 servo4 = Servo(servo4_pin, pin_factory=factory)
21 servo5 = Servo(servo5_pin, pin_factory=factory)
```

Figura 6

⁷ <https://gpiozero.readthedocs.io/en/stable/>

nuovo comando. Anche la parte che riguarda il movimento non è cambiata particolarmente, l'ho solamente riordinata e resa un po' più elegante.

3.4 presa oggetti:

Dopo aver finito con il programma del movimento, ero ormai a metà delle vacanze estive, ho deciso dunque di inoltrarmi nella seconda parte del mio lam, ovvero il controllo del braccio da remoto. Per controllare il braccio da remoto pensavo di usare il protocollo cgi⁸, questo protocollo permette di creare un server e anche una pagina dinamica tramite il linguaggio di programmazione python.

Il mio intento era quello di creare un sito web che si interfacciava al Raspberry pi e mi permetteva tramite dei bottoni, creati in html, di muovere tutti i motori del mio braccio. Questa idea però è fallita poiché ho avuto un

problema con la configurazione di apache2, il sistema che permette di "costruire" un sito web. Il mio problema è stato che non sono riuscito a legare un file python a un file html tramite il protocollo cgi, dopo innumerevoli tentativi, continuavo a non riuscire, anche se avevo provato a scaricare e reinstallare tutti i sistemi più volte, così decisi di aspettare il ritorno a scuola e di chiedere al docente.

Nel frattempo per evitare di passare il resto dell'estate a non fare niente, ho cominciato a pensare cosa altro potevo far eseguire al mio braccio, fino a che, navigando un po' in rete, ho visto un video di una catena di montaggio con un braccio che prendeva degli oggetti e li metteva al loro posto.

Dopo aver visto questo video ho deciso di volerli provare anch'io, ma non sapevo come fare. I miei dubbi sono spariti quando, seduto alla scrivania, ho visto un sensore di distanza che avevo usato in precedenza per un progetto di una lampada che si accendeva con un gesto, allora ho preso il sensore di distanza e ho cominciato a pensare a come interfacciarlo con il Raspberry pi.

Il sensore che ho usato per fare il mio sistema di presa automatica è un sensore HC-SR04 Ultrasonic Range Sensor, questo sensore utilizza le onde sonore per trovare la distanza di un oggetto. È composto da due altoparlanti ad alta frequenza, che emettono onde a 40kHz, da un microprocessore per l'elaborazione dei dati e 4 pin, come al solito due di questi pin servono per la corrente di alimentazione e per la messa a terra mentre i due rimanenti sono i pin Trig e il pin Echo, il primo è il pin che riceve il segnale dal computer, e lo tramuta in onda, mentre il pin Echo ha lo scopo di ricevere l'onda indietro e di comunicarla al computer per essere successivamente elaborata.

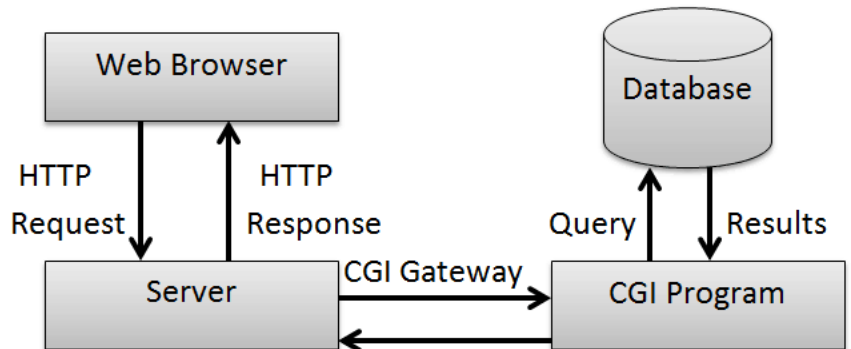


Figura 7: Protocollo CGI

⁸ <https://networkencyclopedia.com/common-gateway-interface-cgi/>

Comunque il funzionamento del sensore lo andrò ad approfondire in seguito con il supporto del codice.

3.4.1 Programma per rilevare la distanza1:

Dopo avere collegato il sensore al Raspberry pi e aver trovato le resistenze ideali sono passato alla scrittura del programma; il programma è molto standard anche se su internet se ne trovano diverse versioni. Non ne ho trovata una che si addiceva perfettamente al mio caso, così sulla base di diversi programmi ne ho elaborato uno io che funzionava perfettamente. Il

programma in questione, visibile a destra, è relativamente semplice, dopo avere dichiarato i pin e la loro funzione bisogna inizializzarli e poi, tramite due loop, continuare a segnare il tempo di emissione e di ricezione delle onde. Dopo aver preso il valore di invio e di ricezione si può calcolare il tempo del percorso compiuto facendo semplicemente il delta T, dopodiché si prende il valore lo si moltiplica per la velocità del suono e si trova la distanza percorsa in totale dall'onda; ora bisogna solo dividere il valore per due ed abbiamo la distanza dall'oggetto.

Per controllare se il programma andava ho messo il sensore su un righello e un oggetto a una determinata distanza e ho letto se il valore

```
24 #settare metodo conteggio pin
25 GPIO.setmode(GPIO.BCM)
26 #settare il numero dei pin del sensore di distanza
27 TRIG = 23
28 ECHO = 18
29 # inizializzare i pin del settore di distanza
30 GPIO.setup(TRIG,GPIO.OUT)
31 GPIO.setup(ECHO,GPIO.IN)
32 #inizializzare il pin Trig
33 GPIO.output(TRIG, False)
34 time.sleep(1)
35 GPIO.output(TRIG, True)
36 time.sleep(0.00001)
37 GPIO.output(TRIG, False)
38 #segnare il momento in cui parte il suono
39 while GPIO.input(ECHO)==0:
40     pulse_start = time.time()
41 # segnare il momento di arrivo dell onda
42 while GPIO.input(ECHO)==1:
43     pulse_end = time.time()
44 # calcolare il tempo di viaggio del suono
45 pulse_duration = pulse_end - pulse_start
46 #calcolare la distanza percorsa
47 distance = pulse_duration * 17150
48 distance = round(distance, 2)
49 print("Distance:",distance,"cm")
50 return distance
51 GPIO.cleanup()
```

Figura 8

trovato era veramente uguale alla distanza. A mio dispiacere non era molto preciso.

3.4.2 Movimento in base alla distanza:

Dopo essere riuscito a installare e a programmare il sensore di distanza sono passato all'implementazione di questo nuovo codice nel programma principale con lo scopo di far raccogliere al braccio un oggetto rilevato sulla superficie. E da qui sono scaturiti diversi problemi. Il primo, che è stato il più semplice da risolvere, anche grazie all'aiuto del mio compagno di Lam Danylo, era un problema di pura programmazione, ovvero non mi ricordavo più come si faceva per far passare delle informazioni tra una funzione e un'altra in python. Ma come già detto, grazie all'aiuto di Danylo, l'ho risolto abbastanza in fretta. L'ho risolto semplicemente facendo ritornare il valore appena calcolato dalla funzione di rilevamento e ugagliare la funzione a una variabile nella funzione di movimento. Adesso andrò brevemente ad illustrare la funzione del movimento e gli altri problemi che ho riscontrato.

La funzione è quella situata qui a destra, le prime 10 linee (tranne la 68 che approfondirò in seguito) servono per mettere in posizione di base il braccio, dopodiché inizia il loop e come si vede continua a prendere il valore dalla funzione `clean()`, che è quella vista in precedenza. Se il valore preso da questa funzione è maggiore di 20 cm allora non succede niente, o meglio il braccio continua ad oscillare avanti e indietro come un radar, mentre se la distanza è inferiore a 20 cm, la rotazione si ferma, il braccio viene però spostato leggermente in avanti, poiché quando il sensore rileva l'oggetto il braccio non è ancora nella posizione ottimale per raccoglierlo, ma è un po' prima.

Dopo che il braccio si è posizionato nella posizione corretta il loop si ferma e in

```

67 def movement():
68     x = 0
69     #posizioni dei motori
70     b = 450
71     c = 100
72     d = 200
73     e = 150
74     f = 150
75     keep = True
76     #comunicare la gradazione a cui devono ruotare i motori
77     servo2.value= math.sin(math.radians(c))
78     servo3.value= math.sin(math.radians(d))
79     servo4.value= math.sin(math.radians(e))
80     servo5.value= math.sin(math.radians(f))
81     while keep:
82         #chiamare la distanza dalla funzione precedente
83         distance = clean()
84         #se la distanza è maggiore di 10 il motore ruota
85         if distance >= 20:
86             b += 5
87             servo1.value= math.sin(math.radians(b))
88
89         #se no si ferma la rotazione
90         elif distance < 20:
91             x = b + 10
92             servo1.value= math.sin(math.radians(x))
93
94             keep = False
95
96         if distance <20 and distance >= 15:
97             c = 166
98             d = 191
99             e = 105
100            f = 185

```

Figura 9

base alla distanza a cui si trova l'oggetto vengono impostate le coordinate a cui i motori devono mettersi per raccogliere l'oggetto; questa è la versione definitiva, poiché anche qui ho incontrato un paio di problemi nella programmazione, ma nessuno degno di nota.

I valori a cui il braccio deve muoversi sono del tutto sperimentali, con questo intendo che ho fatto diversi esperimenti per diverse distanze, più o meno un caso per ogni 4 cm di distanza, e ho spostato il braccio (con il programma del movimento con la tastiera) fino a prendere l'oggetto; e poi ho segnato la distanza. Questo metodo seppur rudimentale avrebbe dovuto funzionare, ma a mio stupore non è mai funzionato. Infatti quando lanciavo il programma tutto sembrava andare, ma dopo un po' il sensore di movimento smetteva di fornire dati e di conseguenza il braccio si fermava. Questo problema è stato uno dei più difficili da risolvere, soprattutto perché il mio codice è giusto, non c'è nessuno problema di programmazione. Dopo aver passato molto tempo a cercare su internet il motivo del mio problema, ho deciso di provare a chiedere su un forum se qualcuno sapesse il motivo di questo mal funzionamento. Così mi sono iscritto al forum dedicato al raspberry pi⁹ e ho posto il mio quesito. Dopo un ora **B.Goode** aveva già risposto, dicendomi che il problema derivava dall'utilizzo di due diverse librerie, o meglio dal fatto che il comando `GPIO.cleanup()` a linea 51 viene usato dalla libreria `RPI.GPIO`, quella che ho usato per trovare la distanza, e questo comando va ad interferire con la libreria che uso per il movimento dei motori, e questa interferenza tra le due librerie ha causato il malfunzionamento del braccio.

⁹ <https://forums.raspberrypi.com/>

Così ancora una volta ho dovuto riscrivere il programma sostituendo la libreria Rpi.GPIO con la libreria gpiozero.

Le modifiche però non mi hanno preso troppo tempo,

```
25 from gpiozero import DistanceSensor
26 echo_pin = 7
27 trig_pin = 23
28 sensor = DistanceSensor( echo = 7, trigger = 23, pin_factory = factory)
29
```

Figura 11

anche perché la libreria gpiozero ha già una funzione preesistente per l'utilizzo dei sensori di distanza; tramite l'utilizzo di questa libreria ho anche risolto, mediante la funzione factory, il problema della precisione delle misure e anche il fatto che quando il sensore vedeva l'oggetto il braccio non era nella posizione corretta. Grazie alla funzione già preesistente per il sensore di distanza anche il codice per il sensore è molto più semplice, infatti mentre stavo provando a riscrivere il programma con la nuova libreria ho scoperto che nella documentazione ufficiale veniva fornito il codice per l'utilizzo del sensore di distanza come visibile qui a destra ; perciò ho preso quello e l'ho implementato nel mio codice.

```
158 def clean():
159     distance = round((sensor.distance* 100), 2)
160     print('Distance: ', distance, "cm")
161     time.sleep(0.01)
162     return distance
```

Figura 10

Il resto del codice, ovvero la funzione movement() è rimasta quasi invariata, ho cambiato solo due cose: la prima è che ho rimosso la parte in cui dicevo al braccio di muoversi leggermente in avanti dopo aver trovato un oggetto e la seconda è che dopo aver trovato l'oggetto al posto di fermare subito le misurazioni ne fa altre cinque a intervalli di 0.1 secondi, ho fatto questo perché così il braccio fa in tempo a stabilizzarsi e la misura risulta molto più precisa.

3.4.3 Movimento in base alla trigonometria:

Dopo essere riuscito a far prendere al braccio gli oggetti senza difficoltà è successa una cosa drammatica. Un giorno mentre stavo sperimentando il nuovo programma appoggiando degli oggetti sulla scrivania e facendoli raccogliere dal braccio ho dimenticato di spostare la breadboard con tutti i cavi fuori dal suo campo visivo, questo è stato uno sbaglio che avrei pagato caro, infatti avevo posizionato l'oggetto troppo lontano dal sensore e il braccio non l'ha visto e continuando il suo percorso si è trovato davanti la breadboard con i suoi cavi, così si è fermato, e si è abbassato a prenderla, si è attaccato ai cavi e li ha strappati tutti! Dopo di questo il braccio è caduto morto a terra perché non aveva più alimentazione. Questa è la drammatica storia del suicidio del mio braccio e di come ho passato le ore seguenti a riattaccare tutti i cavi!

Il codice del rilevamento degli oggetti, o meglio il codice del movimento dopo l'aver trovato un oggetto, però non mi soddisfaceva. Questa delusione derivava dal fatto che il mio braccio per andare a prendere gli oggetti andava a tentativi, ovvero si muoveva in base a differenti casi e perciò gli angoli sono sempre gli stessi, mentre a me sarebbe piaciuto avere una formula che mi avrebbe permesso di calcolare gli angoli per ogni specifica distanza. Per risolvere questo problema ho pensato di poter usare la trigonometria; questo poiché ho notato che quando il braccio si abbassava a prendere un determinato oggetto si creava un triangolo, con due lati composti dai due avambracci e il terzo lato formato dalla distanza tra la base del braccio e l'oggetto. Da qui ho pensato che, tramite il teorema del coseno, sarei riuscito a risalire alle angolazioni; così sono riuscito a trovare una formula che mi ha permesso di calcolare gli angoli per ogni oggetto localizzato grazie al sensore di distanza. Queste sono

le linee di codice che mi permettono di aggiustare gli angoli per le date distanze. Comincio partendo dalla creazione di due variabili che saranno poi i valori dei miei angoli, poi dichiaro le distanze fisse, quelle dei due avambracci e imposto anche la variabile della distanza tra sensore ed oggetto; a questa distanza aggiungo tre centimetri, poiché il sensore è attaccato all'esterno del motore e non nel suo centro.

Anche le altre due distanze non sono uguali a quelle dei due avambracci in legno, poiché devo considerare anche la parte in metallo attaccata al perno di rotazione dei motori.

```

212     alpha = 0
213     beta = 0
214     arm1Lenght = 25
215     arm2Lenght = 32
216     distance1 = distance1 +3
217     alpha = math.acos((distance1**2+ arm1Lenght**2 -arm2Lenght**2)/(2*distance1*arm2Lenght))
218     beta = math.acos((arm2Lenght**2+ arm1Lenght**2 -distance1**2)/(2*arm1Lenght*arm2Lenght))

```

Figura 12

Dopo aver trovato tutte e tre le misure il mio programma esegue due serie di operazioni, queste due serie di operazioni sono basate sul teorema del coseno, infatti le operazioni sono solamente il teorema del coseno con l'angolo isolato.

Faceno questi calcoli il mio programma arriva a capire l'angolazione necessaria dei miei motori. Rimaneva però un problema perché io conoscevo i gradi dell'angolo che si doveva formare tra i due avambracci ma non sapevo se impostando quell'angolo ai motori si sarebbero messi nella posizione corretta perché magari l'angolo che si formava era uno, ma l'angolo a cui il motore ruotava era un altro. Perciò ho fatto un po' di test per trovare una correlazione tra l'angolo del triangolo e l'angolazione del motore (tutto in modo empirico).

I risultati dopo i numerosi test sono questi:

```

220     servo2.value= math.sin(360 -math.radians(math.degrees(alpha)))
221     servo3.value= math.sin(math.radians(155 +math.degrees(beta)))

```

Figura 13

dal programma si può vedere che il motore tra i due avambracci ha una posizione di base di 155 e perciò sommando l'angolo a 155 (ovviamente convertendo beta in gradi) si trova l'angolazione corretta. Per alpha è stato invece parecchio più complicato, infatti anche lì analogamente a beta stavo cercando il valore base a cui sommare l'angolo per arrivare all'angolazione a cui ruotarlo, ma dopo molti tentativi non riuscivo ancora.

Ma a un tratto ho provato a mettere 360 meno l'angolo base, perché pensavo che magari se faceva un giro completo poi si posizionava in maniera corretta, ma per sbaglio al posto di sottrarre a 360 il valore dell'angolo in gradi ho sottratto quello in radianti, ho fatto partire il codice ed è magicamente funzionato. Il motivo per cui funziona non lo so, detto sinceramente per me non ha alcun senso, poiché mischio un'unità di misura in radianti con una in gradi; ma francamente ci stavo provando da così tanto tempo che ho applicato la regola del "se funziona non toccarlo più" e perciò il programma è ancora così adesso.

3.5. Mark II e il suo nuovo volto:

Ormai entrambi i programmi funzionavano senza problemi, decisi così allora di migliorare il braccio a livello meccanico, soprattutto perché volevo evitare un suo altro suicidio perché l'ultimo mi era costato ore e ore di lavoro nel ricollegare tutti i cavi.

Decisi così di costruirgli una “casa” in cui poter mettere la scheda breadbord e tutti i cavi in modo sicuro e senza pericolo che il braccio li sradichi di nuovo.

All’inizio pensavo di costruire una scatola di legno da zero partendo da delle lastre di compensato, così ho preso la bici e sono andato alla Migros a comprarle, ma quest’idea è stata un fallimento. Infatti dopo aver, con molta fatica, costruito la scatola ho montato il braccio al suo interno, la base era attaccata alla parete con due viti, ma il problema era l’eccessivo peso del braccio, infatti poco dopo averlo attaccato alla scatola il compensato ha ceduto e la scatola è collassata. Così dopo questo fallimento ho optato per utilizzare una scatola più resistente, magari fatta da qualcuno più abile di me. La scatola perfetta l’ho trovata in cantina ed è una scatola per le bottiglie di vino, infatti questa scatola è molto resistente e anche salda. Così ho passato tutto il giorno seguente a tagliare e colorare la scatola fino ad averla delle dimensioni giuste e anche con dei buchi dai quali far entrare i cavi per il Raspberry pi.

Assieme a questa scatola ho anche costruito, partendo da una lastra di compensato, un piano a livello della scatola che funge da base su cui appoggiare gli oggetti da raccogliere. Dopo aver pitturato la scatola di rosso all’esterno e di bianco all’interno con le bombolette spray (anche se poi successivamente l’ho ridipinto di nero) ho deciso di vivacizzare il braccio pitturandolo tutto di rosso e aggiungendoci due occhi che si muovono.

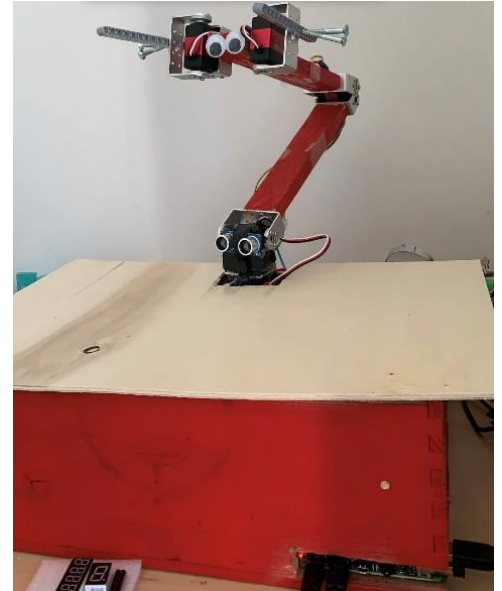


Figura 14: Mark II

3.5.1 il nuovo volto 2.0:

Adesso mi permetto un salto temporale, infatti salto dalla fine dell’estate fino metà novembre, in questo periodo ho però continuato a scrivere codici soprattutto per quanto riguarda il controllo da remoto ma ho deciso di parlare prima del design visto che ho già iniziato a trattare il tema così da non dividere il tutto a metà.

Il design ha avuto un netto upgrade quando, per il mio compleanno, ho ricevuto in regalo una stampante 3D, dopo averla configurata e aver stampato un paio di progetti di prova, ho deciso di stampare dei rimpiazzati per gli avambracci in legno.

Il vero motivo per cui l’ho fatto, oltre all’estetica, è che stampandoli in 3D avrei potuto costruire delle scanalature esterne in cui fare passare i cavi così da non avere più cavi attaccati male con lo scotch che continuavano a dare fastidio e ad impigliarsi.

Così dopo un po’ di tutorial sull’utilizzo del programma di modellizzazione ho, con molta fatica, disegnato e successivamente stampato i due nuovi pezzi.

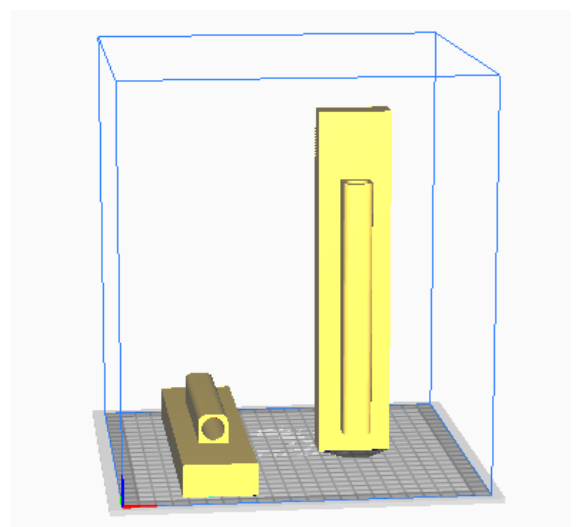


Figura 15: Stampa 3D avambracci

Il disegno l'ho fatto su un programma online chiamato onshape ¹⁰. Dopo averli stampati è arrivata una delle parti peggiori, ovvero dovevo togliere tutti i motori e riattaccarli ai nuovi pezzi, ma soprattutto dovevo collegare tutti i cavi singolarmente che prima erano collegati con degli appositi connettori a pin maschio e femmina che però ogni tanto non facevano contatto. Bisognava quindi farli passare attraverso le scanalature e isolarli con le fascette termorestringenti. Questo lavoro mi ha occupato per un weekend intero, anche perché in molti casi ho dovuto aggiungere perfino dei pezzi di cavi in più perché erano troppo corti. Dopo aver fatto tutto questo lavoro è successa una delle cose peggiori che può succederti quando hai tanti cavi, ovvero un cavo del sensore di distanza, precisamente quello del pin TRIG, si è spostato di una posizione nella breadbord, e perciò non permetteva il passaggio di segnale dal computer al sensore.

Per risolvere questo problema ci ho messo letteralmente due settimane e mezzo, ho detto che questo è uno dei peggiori perché non c'è un errore visibile subito, anzi spesso sfugge alla vista, perciò prima ho passato un sacco di tempo sul programma e poi sono andato a ricostruire tutti i singoli collegamenti fino a quando non ho trovato il collegamento sbagliato. Dopo aver risolto anche questo problema ho deciso di ridisegnare anche le pinze in 3D, che fino a quel momento sono state un pezzo di lego attaccato con dello scotch ai supporti in alluminio del motore. Il disegno delle pinze non l'ho però fatto io, l'ho preso da un sito in cui si pubblicano progetti¹¹, avevo già sofferto abbastanza con quello delle braccia! Io ho solo disegnato dei piccoli cilindri che coprivano le viti che usavo per tenere insieme la pinza superiore ed inferiore. Per chiarire un po' le cose appena dette è d'obbligo spiegare come ho fatto le nuove pinze. Esse sono delle specie di mezzelune con la fine larga per permettere una maggior superficie di presa, di queste pinze ne uso due con dei motori, infatti ne collego una al perno di rotazione superiore e la seconda al perno di rotazione inferiore in verso opposto, per aumentare così ancora la superficie di presa.

Dopodiché ho collegato le due pinze di un motore tra loro con delle viti, che ho dovuto tagliare con la perles e poi ho coperto le viti con dei cilindri stampati in 3D per rendere il tutto più bello esteticamente. Ho dovuto collegare le due pinze perché solo uno dei due perni di rotazione viene mosso dal motore, l'altro è libero e perciò deve essere mosso, e quindi collegato, da quello superiore. Per rendere il movimento della pinza inferiore uguale a quella superiore ho usato, come già detto, delle viti; per rendere i movimenti analoghi ne ho dovute però usare due, una vicino alla base e la seconda vicino all'estremità.

Ho dovuto fare questo processo per entrambi i motori. Al nuovo braccio ho però sempre aggiunto i due occhi e questa volta anche un cappello stampato in 3D e così è nato Mark III



Figura 16: Mark III

¹⁰ <https://www.onshape.com/>

¹¹ <https://www.thingiverse.com/thing:3648782>

4 controllo remoto:

4.1 controllo da website:

Ho deciso di non dilungarmi troppo parlando di questa parte questo perché, anche se mi ha occupato tanto tempo, non è mai andata molto avanti. La causa di ciò è il protocollo cgi.bin che sebbene funzioni molto bene per imparare l'hosting di website e ti permette di controllare pagine dinamiche da file in python, rimane comunque uno strumento abbastanza debole e con poche potenzialità, ma soprattutto il più grande problema è il fatto che non viene usato da nessuno ad alti livelli; e perciò ci sono delle grandi lacune nella documentazione, vengono detti pochi comandi e soprattutto non vengono spiegati in modo esaustivo. Ho anche formulato diverse domande sul forum del Raspberry pi, ma nessuno è mai riuscito a darmi una risposta soddisfacente per me. Ho deciso comunque di ripercorrere un po' il percorso fatto.

Ho cominciato il controllo dopo che, grazie al docente, ero riuscito a far comunicare il file python e quello html, su piccola scala, ovvero controllando tramite un singolo pulsante l'accensione e lo spegnimento di un led. Solamente per questo programma, che alla fine è molto basico, ho passato ore e ore a guardare link su link per trovare qualcuno che utilizzasse il protocollo cgi per controllare un led, in generale utilizzano tutti un file html reso dinamico da uno in Java.

Non ricordo più bene come ma ho trovato il comando `form.getvalue()`, questo comando andava legato al file html inserendo all'interno delle parentesi il valore del pulsante di tipo "submit" che si voleva usare, quando quest'ultimo veniva premuto il comando sopra passava da 0 a 1 e perciò si poteva usare per accendere o spegnere un led.

Rimaneva ancora un problema, ovvero che il bottone usato nel file html era un pulsante di tipo "submit", perciò dopo essere stato premuto e aver comunicato il tutto al file in python necessitava di essere reindirizzato ad una nuova pagina.

Altrimenti usciva il solito "INTERNAL SERVER ERROR" come da consueto.

Tra l'altro ho scoperto solamente verso la fine del mio percorso nel creare il sito web, che potevo sapere che errore era presente quando usciva "INTERNAL SERVER ERROR" andando in una determinata cartella. Perciò dopo aver eseguito il programma di python bisognava "riprintare", traducendo in linguaggio di html tramite uno specifico comando, il file html.

Da qui in avanti il mio programma non è più cambiato molto (anche se ci ho passato moltissimo tempo sopra) infatti ho solamente sostituito il comando per accendere un led con quello per muovere un singolo motore preso dal codice del movimento della tastiera e aggiunto anche due bottoni per ogni motore. Questi sono stati i cambiamenti nel file python, mentre quelli del file html sono stati più che altro cambiamenti di design tramite l'utilizzo del linguaggio di programmazione Java, di cui ho capito le basi per fare i bottoni più belli o aggiungere una foto come sfondo.

Questo è il file python per il movimento:

```
35 print( "Content-type: text/html\n")
36 form = cgi.FieldStorage()
37 #check wich button is pressed and store the new servo position
38 if form.getvalue("sinistra1"):
39     b += speed
40     if b > 440 :
41         b = 440
42 if form.getvalue("destra1"):
43     b -= speed
44     if b < 290:
45         b = 290
```

Figura 17

4.1.1 salvataggio posizione:

L'unica parte interessante di questo programma è il modo in cui salvavo le posizioni dei motori, infatti siccome il file riparte ogni volta che un bottone viene premuto non potevo far ricordare al programma stesso i dati delle posizioni dei motori aggiornate, ma dovevo trovare un modo per salvare i file esternamente al programma quando esso era stato eseguito, per poi riprenderli quando veniva eseguito di nuovo.

Dopo svariati tentativi per fare questo sono ricorso, grazie all'aiuto del docente, all'utilizzo di un file di testo. Infatti ciò mi permetteva di salvare i dati della posizione del motore esternamente al programma e anche di riprenderli. Il processo con cui faccio questo lo spiegherò meglio quando parlerò della connessione tramite ssh che è andata a rimpiazzare il controllo da remoto del sito web. A questo punto il codice del sito web locale funzionava, infatti io potevo navigare all'indirizzo ip della rete connessa e comandare il braccio con i pulsanti. Il problema era però il fatto che i pulsanti, quando venivano premuti, muovevano il braccio da fermo di una decina di gradi, e questo movimento improvviso faceva oscillare molto tutto il braccio, perciò il controllo era scostante e anche leggermente caotico. Un altro problema era che il protocollo cgi-bin era abbastanza lento ad elaborare la richiesta, perciò spesso, e soprattutto quando si premevano i pulsanti a brevi intervalli il movimento del braccio era davvero pessimo. Perciò ho deciso di tentare di far muovere i motori in un altro modo, ovvero quando il pulsante veniva premuto per la prima volta questo faceva cominciare a ruotare il motore e quando veniva ripremuto si fermava.

Non sono mai riuscito a fare questa cosa, soprattutto per colpa dei bottoni di tipo "submit" che mi facevano ogni volta riaggiornare la pagina. Ho fatto anche molti altri tentativi per migliorare il controllo, come cambiare il tipo di user input, ma tutti vani.

Decisi così di provare a controllarlo in un altro modo, ovvero di controllarlo con una connessione ssh.

4.2 connessione ssh:

Tramite la connessione ssh volevo semplicemente realizzare l'idea del movimento continuo fin quando un pulsante lo ferma, l'ho spiegato meglio sopra. E necessario però, prima di iniziare, spiegare cosa è una connessione ssh (o secure shell), una connessione ssh è: "SSH, also known as Secure Shell or Secure Socket Shell, is a network protocol that gives users, particularly system administrators, a secure way to access a computer over an unsecured network"¹². Essa è un protocollo con cui un utente può accedere da remoto a un computer e eseguire dei comandi su di esso in modo sicuro. Io ho usato questa connessione per eseguire un comando di esecuzione di un codice di movimento. Per interfacciarmi tramite il protocollo ssh al Raspberry pi ho usato l'app SSH remote per ios, al cui interno si potevano creare dei SSH button, ovvero dei bottoni che quando premuti eseguivano un'operazione da remoto sul computer. Prima di guardare che operazione eseguivano spiego il programma che ho caricato nei ssh button. All'inizio come al solito ho dovuto prendere un po' di confidenza con questo sistema, così ho sviluppato un piccolo programma che creava un file e scriveva al suo interno

¹² <https://www.techtarget.com/searchsecurity/definition/Secure-Shell>

“ciao”, dopo un po’ di tentativi falliti sono riuscito a far funzionare questa cosa. Così sono passato a fare un programma per il movimento, il programmmaramma che ho creato è derivato sia dal programma della tastiera che dal programma del web server, infatti anche questo, come quello della tastiera, è bastao su un loop che aumenta una variabile da inserire poi all’interno del comando che sposta i motori, ma la differenza è che non è presente un solo programma per controllare tutti, ma un programma equivale ad un pulsante del sito web, perciò per ogni motore mi servivano due programmi. Siccome ho più programmi per muovere i motori devo trovare un modo per mettere in comune tutti i dati che riguardano la posizione del motore, così da permette la posizione sempre aggiornata per ogni nuovo programma. Fare questo non è stato però un grosso problema, in quanto avevo già dovuto fare una cosa simile con il sito web, la prima beta dell’assunzione dei dati dal foglio era questa: all’inizio aprivo il file locato nella stessa cartella del programma e lo rendevo leggibile, poi usavo la funzione `dati.readline` per leggere il dato scritto sulla linea, i numeri all’interno delle parentesi simboleggiano i caratteri da prendere, poi trasformavo il valore in `int` e questo mi dava le coordinate di partenza del motore. E questo lo facevo con tutti. Però qui nacque un problema, infatti ogni tanto quando faceva partire il programma mi dava questo: `ValueError: invalid literal for int() with base 10: ''`.

```

1  dati = open('dati.txt', 'r')
2  b = int(dati.readline(123))
3  c = int(dati.readline(456))
4  d = int(dati.readline(789))
5  e = int(dati.readline(101112))
6  f = int(dati.readline(131415))
7  dati.close()

```

Figura 18

in pratica mi diceva che uno dei dati che leggeva non poteva essere convertito in `int`, ho passato davvero molto tempo a cercare di capire perché non andava, fino a quando non ho chiseto al docente, infatti quando ho chiesto al docente e abbiamo provato a stampare i dati scoprendo così che mi scriveva il dato della prima linea una riga vuota poi la seconda poi una riga vuota e così via e abbiamo capito che il problema derivava dal fatto che una linea bianca veniva letta dalla funzione e provata ad essere convertita in `int`, ma ovviamente questo non era possibile.

Io non avevo mai notato questo dettaglio prima perché con l’altro compiler che usavo non mi scriveva le linee vuote. Quindi a partire da qua punto siamo arrivati ad elaborare un codice che mi permetteva di prendere i valori escludendo le linee ovvero quello qui sul fianco. Analogamente all’altro prima si apriva il file, solo che qua bisognava dire il suo percorso, poi si crea una lista vuota, in seguito si crea un loop che inserisce nella lista i valori tranne le linee vuote, simboleggiate dallo “\n”(a capo), e in seguito trasformate in `int`. Da qui bastava associare ad ogni variabile di movimento il nuovo valore zero e poi chiudere il file.

```

17  file = open('/usr/lib/cgi-bin/dati.txt')
18  A = []
19  for i in file:
20      A.append(int(i.split("\n")[0]))
21  b = A[0]
22  c = A[1]
23  d = A[2]
24  e = A[3]
25  f = A[4]
26  file.close()

```

Figura 19

Dopo aver preso i valori base dei motori posso usarli per muovere il braccio, il codice è quello riportato qui a destra. Il programma come già detto consiste in un loop che fa aumentare una variabile che a sua volta sposta il braccio, e poi scrive il nuovo valore nel file.txt. Ora rimaneva solo da fare che un altro programma, che sarebbe diventato il bottone di arresto, stoppasse questo loop, prima ho pensato di fare un programma che semplicemente si sovrascriveva a quello del movimento, ma non funzionò. ha funzionato.

Così mi venne venuta l'idea di usare il file.txt che avevo già usato in precedenza. Ho pensato che potevo creare una variabile che sarebbe stata la

condizione che permetteva al loop di andare, e quest'ultima sarebbe stata decisa da il programma d'arresto; quest'ultimo valore sarebbe stato scritto su il file, e ad ogni ciclo ci sarebbe stata una variabile che prendeva il suo valore e verificava se il loop poteva andare o no. Il codice finale è di conseguenza questo a destra.

In se è praticamente uguale al precedente, l'unica cosa è che il loop si verifica solo se la variabile stato è uguale a uno, e la variabile stato viene aggiornata ad ogni nuovo ciclo leggendola dal file, siccome questa variabile veniva messa nella seconda riga per leggere solo questa mi bastava creare la lista e leggere il sesto valore, quindi quello con index 5. Dopodiché il file veniva chiuso e il loop ricominciava. A condizione che la variabile presa fosse stato 1. l'unico modo per fare cambiare la variabile stato è tramite il programma d'arresto, infatti quest'ultimo ha il solo compito di rendere stato uguale a zero e di scrivere il suo nuovo valore all'interno del file.txt.

```

46 while True:
47     b += speed
48     time.sleep(0.1)
49     c = c
50     d = d
51     e = e
52     f = f
53     if b > 440 :
54         b = 440
55         print("massimo raggiunto")
56     servo1.value= math.sin(math.radians(b))
57     servo2.value= math.sin(math.radians(c))
58     servo3.value= math.sin(math.radians(d))
59     servo4.value= math.sin(math.radians(e))
60     servo5.value= math.sin(math.radians(f))
61     dati = open('/usr/lib/cgi-bin/dati.txt','w')
62     dati.write("%s\n" % str(b))
63     dati.write("%s\n" % str(c))
64     dati.write("%s\n" % str(d))
65     dati.write("%s\n" % str(e))
66     dati.write("%s\n" % str(f))
67     dati.write("%s\n" % str(stato))
68     dati.close()
69

```

Figura 20

```

48 while stato == 1:
49     b += speed
50     time.sleep(0.1)
51     c = c
52     d = d
53     e = e
54     f = f
55     print(b)
56     if b > 440 :
57         b = 440
58         print("massimo raggiunto")
59     servo1.value= math.sin(math.radians(b))
60     servo2.value= math.sin(math.radians(c))
61     servo3.value= math.sin(math.radians(d))
62     servo4.value= math.sin(math.radians(e))
63     servo5.value= math.sin(math.radians(f))
64     file = open('/usr/lib/cgi-bin/dati.txt')
65     A = []
66     for i in file:
67         A.append(int(i.split("\n")[0]))
68     stato = A[5]
69     file.close()
70     dati = open('/usr/lib/cgi-bin/dati.txt','w')
71     dati.write("%s\n" % str(b))
72     dati.write("%s\n" % str(c))
73     dati.write("%s\n" % str(d))
74     dati.write("%s\n" % str(e))
75     dati.write("%s\n" % str(f))
76     dati.write("%s\n" % str(stato))
77     dati.close()
78
79

```

Figura 21

```

31 stato = 0
32 dati.write("%s\n" % str(stato))

```

Figura 22

Dopodiché è sorto un nuovo problema, perché dopo che la variabile stato sia diventata zero non poteva andare nessun programma, poiché per il momento nessun programma poteva ricambiare il valore di questa variabile. Così ho dovuto aggiungere ad ogni codice, prima dell'inizio del loop delle linee di codice che mi permettono di far ripartire il loop rendendo stato == 1 e così facendo di far ricominciare tutto da capo. Così ho fatto per ogni singolo programma; quindi per tutti e 8 sono arrivato ad avere la possibilità di muovere il braccio da remoto. Solo che si è verificato un contrattempo; infatti per sbaglio ho caricato il programma sbagliato e il braccio è andato a picchiare con molta forza contro la base spaccando i muri dei pezzi stampati in 3D, che erano spessi solamente 1 mm e fissati però con

due viti. Così, per mia gioia, dovetti ristampare entrambi i pezzi con i muri spessi però tre volte tanto, e anche rifare tutti i collegamenti, poiché ho dovuto estrarre tutti i cavi dalle scanalature del braccio rotto e risaldarli nelle scanalature del braccio nuovo, un altro lavoro che mi ha preso troppo tempo.

Siamo finalmente pronti per poter parlare di come controllavo i motori dal telefono, infatti ora che è chiaro il meccanismo di movimento dei singoli motori e come si fermano posso spiegare come faccio a manovrarmi dal telefono. Come già detto usufruiamo degli SSH button, che sono dei semplici bottoni che quando premuti, e con la stessa rete, vanno ad eseguire un comando sul terminale, nel mio caso ogni bottone eseguiva un comando diverso, ma la base della sua configurazione era analoga. Ed era così:

Prima chiedeva il nome con cui si voleva chiamare il bottone, poi l'indirizzo ip della connessione, in seguito la porta di comunicazione, l'username dell'utente che usa il Raspberry pi e la sua password, ed infine il più importante, ovvero il comando da eseguire. Questo aveva una forma più o meno standard, che è quella dell'esecuzione di un file python, cioè `python3 nomeprogramma.py`, e configurando tutti i bottoni con tutti i movimenti diversi usciva una schermata come quella presente qui a lato.

Finalmente avevi finito il controllo da remoto ora potevo con un pulsante far cominciare la rotazione di un qualsiasi motore in entrambe le direzioni, e poi potevo fermarla con il pulsante d'arresto, ovvero quello "stop". E così, anche se non come avevo immaginato, il controllo da remoto era finito.

Name button
ip
22
username
password
full command

Figura 23: Configurazione dei pulsanti

servo 1 sinistra
servo 1 destra
servo 2 su
servo 2 giù
servo 3 sinistra
servo 3 destra
apertura pinza
chiusura pinza
stop

Figura 24: Schermata finale

5 conclusioni:

La realizzazione di questo progetto è stata in buona parte piacevole, sicuramente mi ha permesso di imparare molto, ho migliorato di sicuro le mie competenze di programmazione ma non solo, ho imparato ad interfacciarmi con una nuovo stile di programmazione per costruire robot o simili, ovvero tramite il modulo Raspberry pi.

Ho appreso anche come funziona l'hosting di siti web in generale e anche ad interfacciarmi da remoto tramite un protocollo a me prima sconosciuto, ovvero il Secure Shell.

Diciamo che ci sono stati comunque dei momenti in cui volevo solo rompere tutto, come negli innumerevoli errori che ho illustrato e anche in altri che ho saltato poiché erano gli stessi ma che si ripresentavano, come quando ho passato una settimana a rifare tutti i collegamenti per cercare il motivo per cui qualcosa non andava più o come quando non capivo perché il mio codice una volta andava e quella dopo no.

Ma tutto sommato è stata una bella esperienza e sono abbastanza soddisfatto del risultato finale.

6 Bibliografia:

<https://en.wikipedia.org/wiki/Servomotor>

https://www.researchgate.net/figure/Schematic-of-an-RC-servo-motor_fig1_328910852

<https://pypi.org/project/RPi.GPIO/>

<https://www.pygame.org/>

<https://gpiozero.readthedocs.io/en/stable/>

<https://networkencyclopedia.com/common-gateway-interface-cgi/>

<https://forums.raspberrypi.com/>

<https://www.techtarget.com/searchsecurity/definition/Secure-Shell>